STEVENS
INSTITUTE OF TECHNOLOGY
1870

PEP 559
Machine Learning in Quantum Physics

Dr. Chunlei Qu

Spring 2026

controlled-NOT (c-NOT)

- The most popular two-qubit entangling gate
- Also known as controlled-X gate
- It acts on two qubits: it flips the 2nd qubit (target) if the first qubit (control) is |1>, and does nothing if the control qubit is |0>

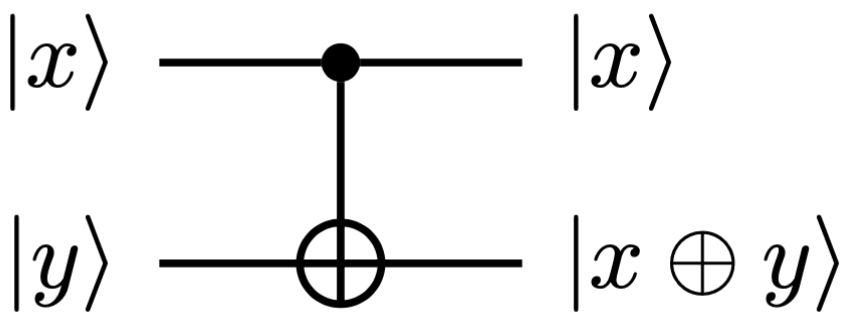
In the standard basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$

$$\left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{array}\right]$$

C-NOT gate circuit notation

control

target

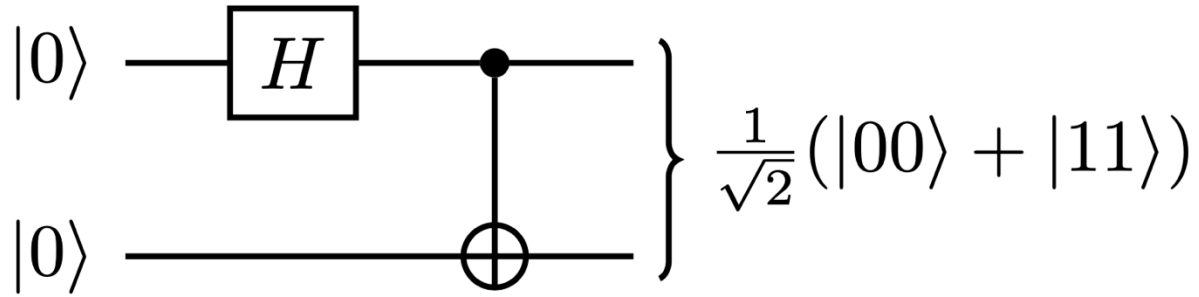


$x, y \in \{0, 1\}$

XOR or addition modulo 2

input		output	
x	y	x	y+x
$\|0\rangle$	$\|0\rangle$	$\|0\rangle$	$\|0\rangle$
$\|0\rangle$	$\|1\rangle$	$\|0\rangle$	$\|1\rangle$
$\|1\rangle$	$\|0\rangle$	$\|1\rangle$	$\|1\rangle$
$\|1\rangle$	$\|1\rangle$	$\|1\rangle$	$\|0\rangle$

c-NOT for entanglement generation



separable

$$|0\rangle|0\rangle \xmapsto{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle$$
$$= \frac{1}{\sqrt{2}}|0\rangle|0\rangle + \frac{1}{\sqrt{2}}|1\rangle|0\rangle$$
$$\xmapsto{\text{c-NOT}} \frac{1}{\sqrt{2}}|0\rangle|0\rangle + \frac{1}{\sqrt{2}}|1\rangle|1\rangle$$

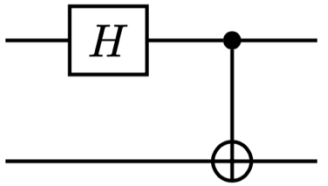
entangled

Quiz:

Try other examples, for example,
If initial state is

|01> → ?
|10> → ?
|11> → ?

Bell states generator



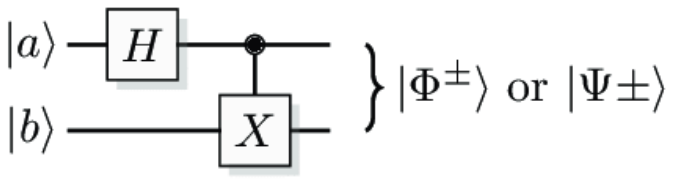
Hadamard + c-NOT gates implements the unitary operation which maps the standard computational basis into the four entangled states, known as Bell states

$$|00\rangle \longmapsto |\psi_{00}\rangle := \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$
$$|01\rangle \longmapsto |\psi_{01}\rangle := \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$
$$|10\rangle \longmapsto |\psi_{10}\rangle := \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$
$$|11\rangle \longmapsto |\psi_{11}\rangle := \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

$$\Phi^+ := |\psi_{00}\rangle$$
$$\Psi^+ := |\psi_{01}\rangle$$
$$\Phi^- := |\psi_{10}\rangle$$
$$\Psi^- := |\psi_{11}\rangle$$

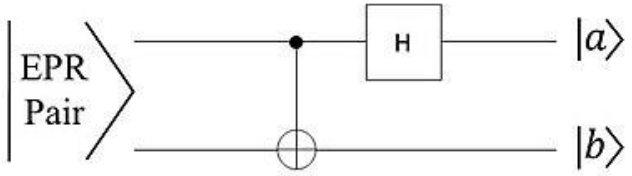
Standard notation for Bell states

Bell states



- They form the **orthonormal basis in the Hilbert space** of two qubits, similar to the original product states basis (computational basis) |00>, |01>, |10>, |11>

- One can also perform measurements in the Bell basis

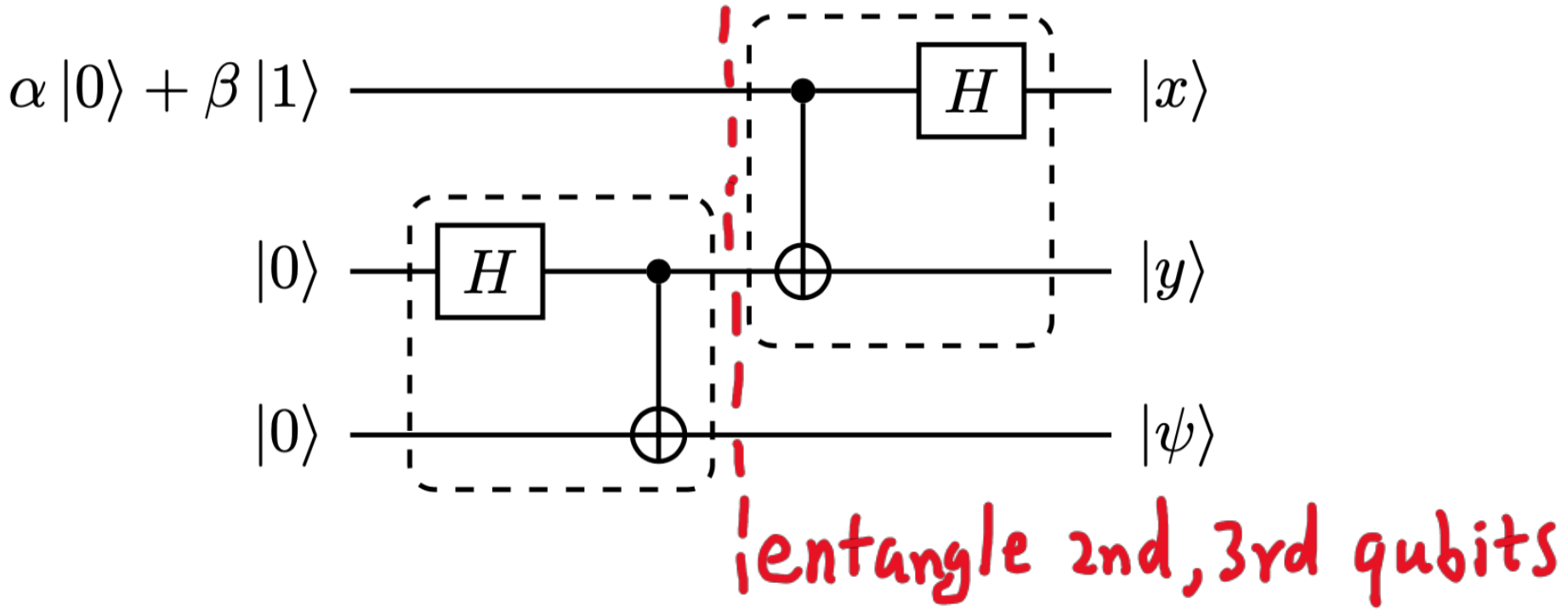


- If we **reverse** the circuit (c-NOT and then Hadamard), it maps the Bell states to the corresponding state |ij> in the computational basis

- The Bell states are **maximally entangled** (their reduced density operators are maximally mixed)

Quantum teleportation

An unknown quantum state can be teleported from one location to another.



A Bell state generator followed by an “offset” inverse Bell state generator

Bell state generator for qubits 2 and 3

After the Bell state generator

$$(\alpha|0\rangle + \beta|1\rangle)(|00\rangle + |11\rangle)$$

It can be re-arranged into another form (for the convenience of applying the following gates)

$$\begin{aligned}
&(|00\rangle + |11\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle) \\
+&(|01\rangle + |10\rangle) \otimes (\alpha|1\rangle + \beta|0\rangle) \\
+&(|00\rangle - |11\rangle) \otimes (\alpha|0\rangle - \beta|1\rangle) \\
+&(|01\rangle - |10\rangle) \otimes (\alpha|1\rangle - \beta|0\rangle)
\end{aligned}$$

Quiz:
Check they are equal

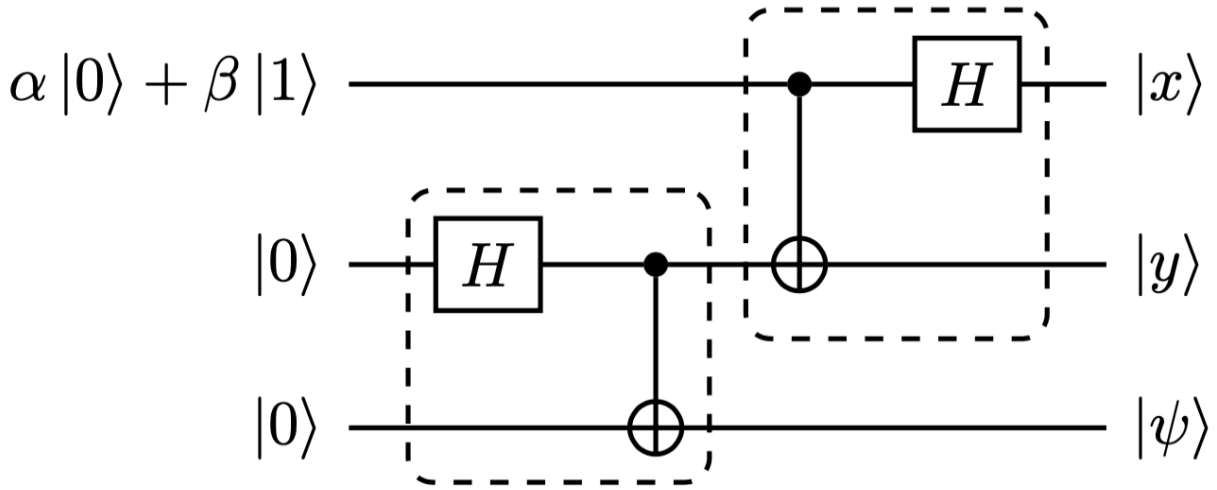
Inverse Bell state generator for qubits 1 and 2

The **inverse Bell state generator** evolves the Bell states (of the first two qubits) back to |ij> in the computational basis

$$
\begin{aligned}
&(|00\rangle + |11\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle) \\
+&(|01\rangle + |10\rangle) \otimes (\alpha|1\rangle + \beta|0\rangle) \\
+&(|00\rangle - |11\rangle) \otimes (\alpha|0\rangle - \beta|1\rangle) \\
+&(|01\rangle - |10\rangle) \otimes (\alpha|1\rangle - \beta|0\rangle)
\end{aligned}
\quad \Rightarrow \quad
\begin{aligned}
&|00\rangle \otimes (\alpha|0\rangle + \beta|1\rangle) \\
+&|01\rangle \otimes (\alpha|1\rangle + \beta|0\rangle) \\
+&|10\rangle \otimes (\alpha|0\rangle - \beta|1\rangle) \\
+&|11\rangle \otimes (\alpha|1\rangle - \beta|0\rangle).
\end{aligned}
$$

Quantum teleportation

Perform standard measurement of the first 2 qubits, i.e., project it to the computational basis



x	y	⇒	$\|\Psi\rangle$
0	0		$(\alpha\|0\rangle + \beta\|1\rangle)$
0	1		$(\alpha\|1\rangle + \beta\|0\rangle)$
1	0		$(\alpha\|0\rangle - \beta\|1\rangle)$
1	1		$(\alpha\|1\rangle - \beta\|0\rangle)$

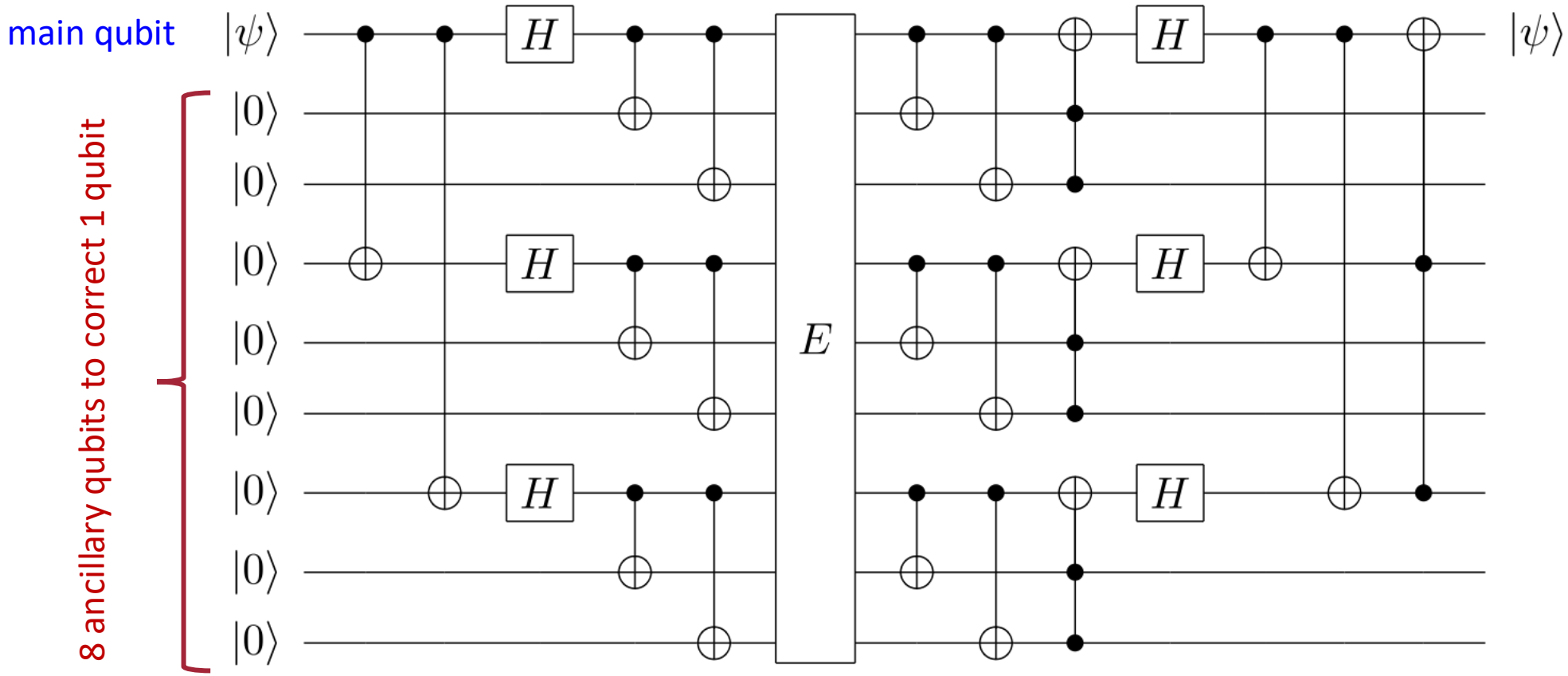
Depending on the measurement outcomes, we know the third qubit will collapse to different quantum state. Then, we can perform another transformation to the 3rd qubit to obtain original state

$$00 \mapsto \mathbf{1} \qquad 01 \mapsto X \qquad 10 \mapsto Z \qquad 11 \mapsto ZX$$

Quantum algorithms

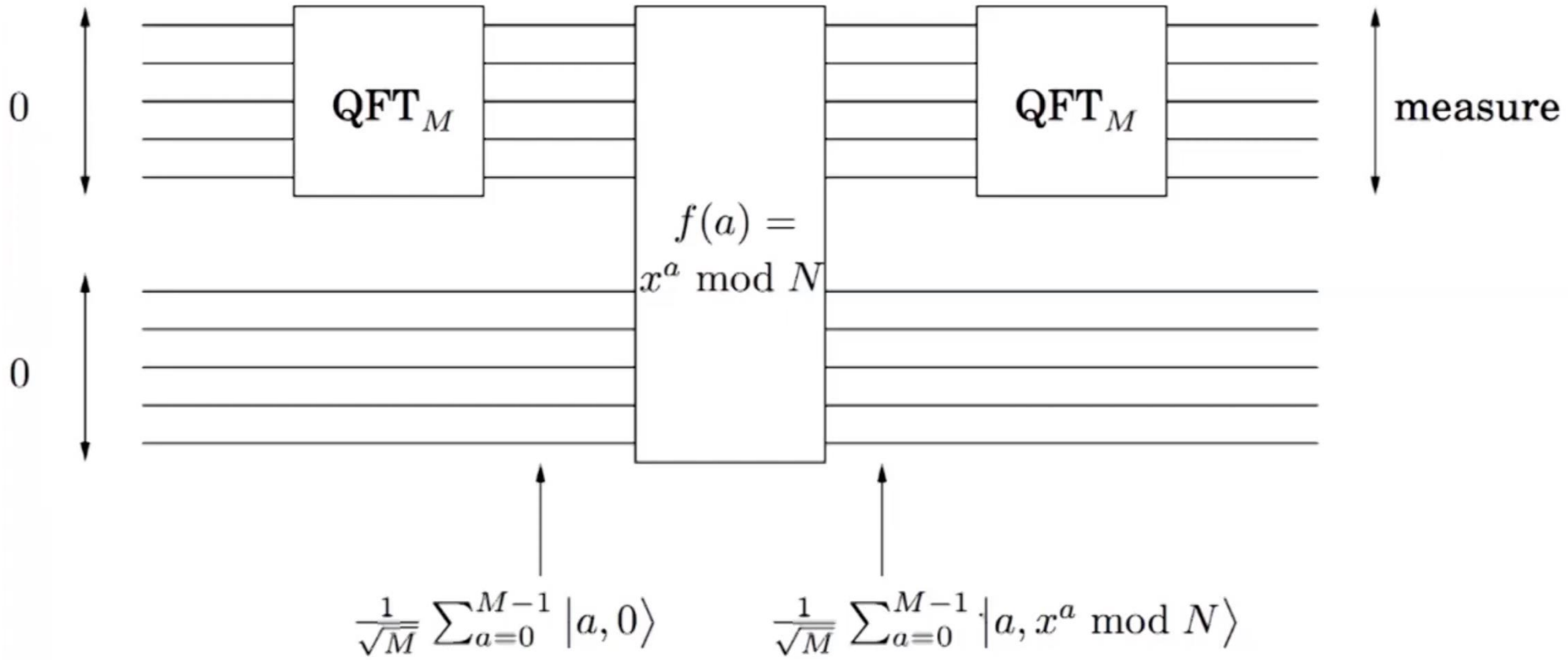
Shor's code for quantum error correction

A specific error correction circuit which can correct both phase flips and bit flip errors

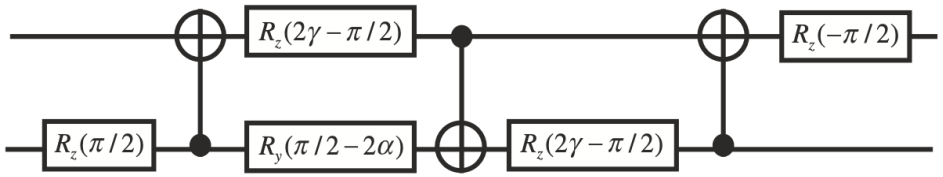


Shor's factoring algorithm

$M > 2N^2$



Question: Quantum gate optimization

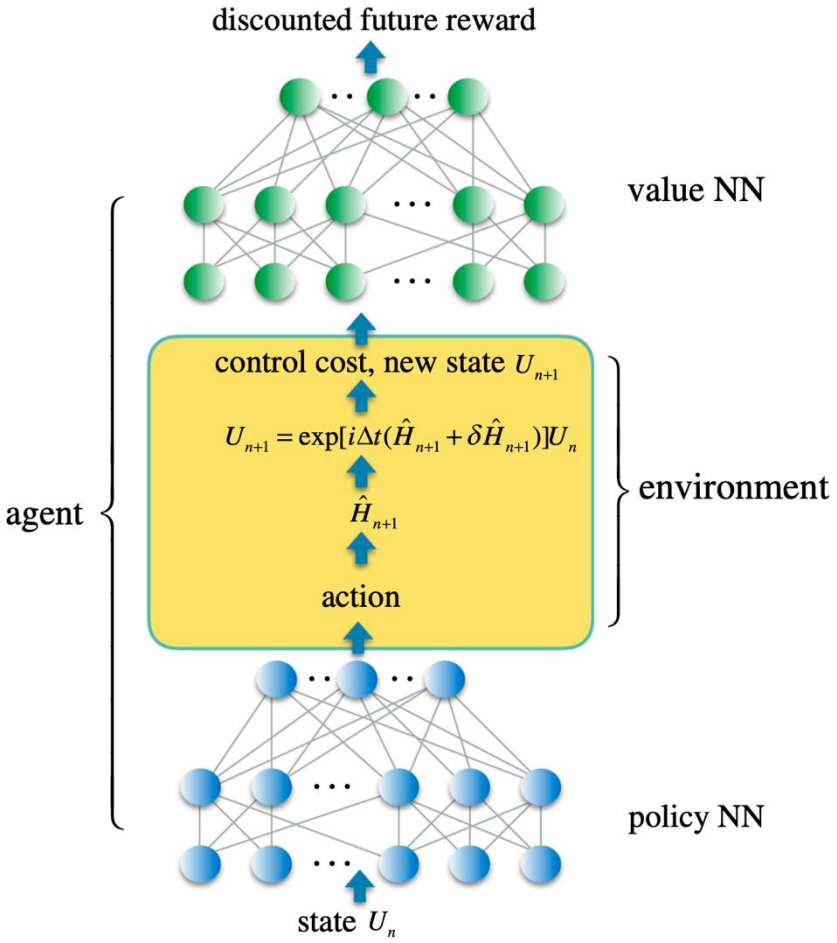


A quantum circuit

- Depth: 7
- Three two-qubit gates
- Five single qubit gates

To realize a unitary transformation

$$\mathcal{N}(a,a,\gamma) = \exp[i(a\sigma_1^x\sigma_2^x + a\sigma_1^y\sigma_2^y + \gamma\sigma_1^z\sigma_2^z)],$$



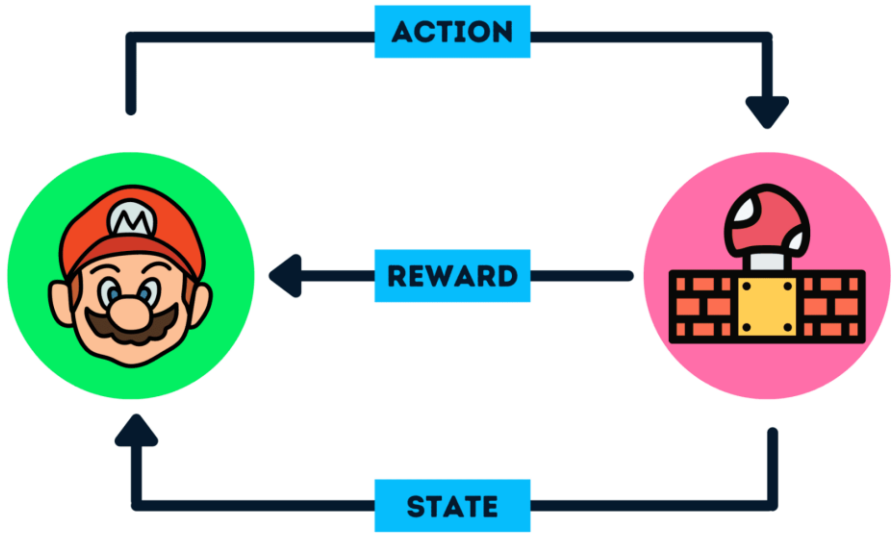
How do we find the optimal parameters to realize such an operation with the above circuit?

Introduction to reinforcement learning

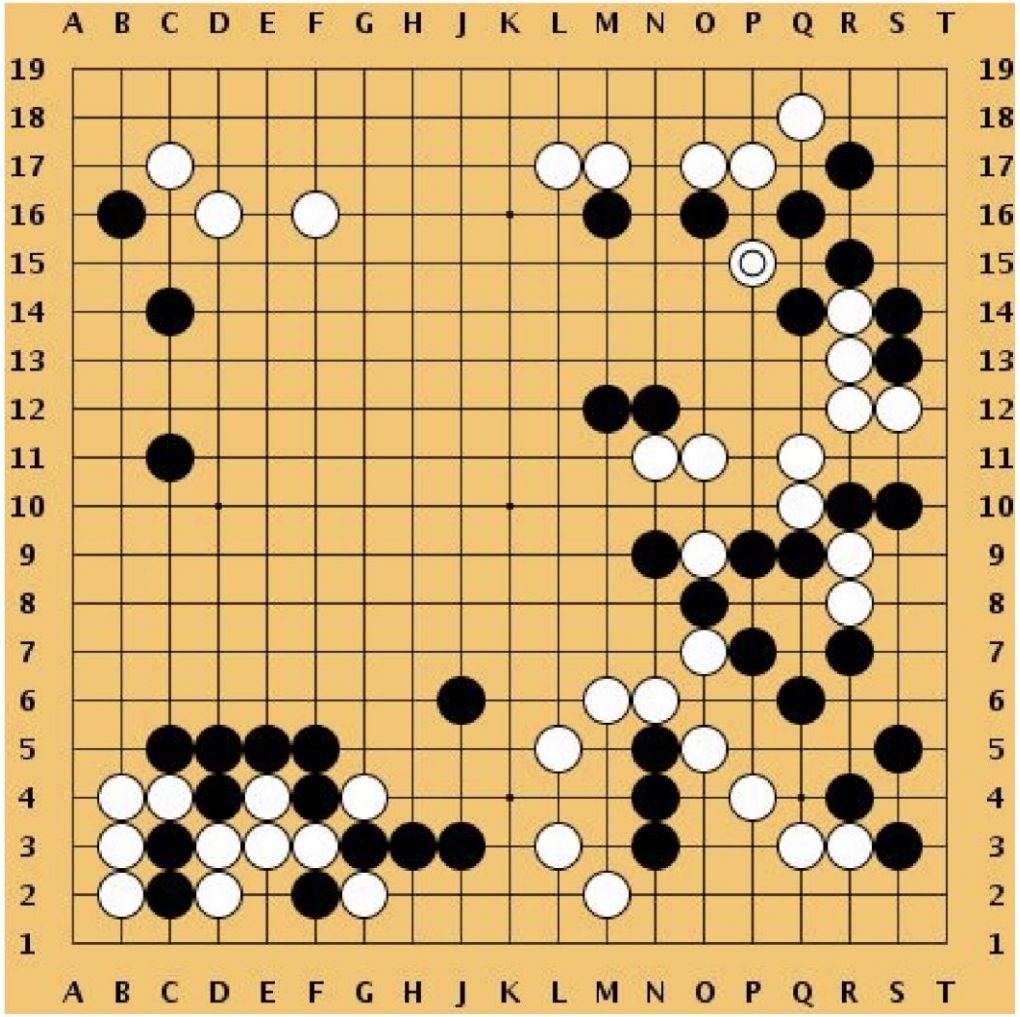
What is Reinforcement Learning (RL)?

- RL is a type of ML where an **agent** learns by interacting with an **environment**
- The agent tries **actions** and receives feedback in the form of **rewards** or penalties.
- The goal is to learn a strategy (**policy**) that maximizes cumulative rewards (*discounted* **return**) over time.
- Unlike supervised learning, RL does not rely on labeled data.

super mario



Board Game: Go



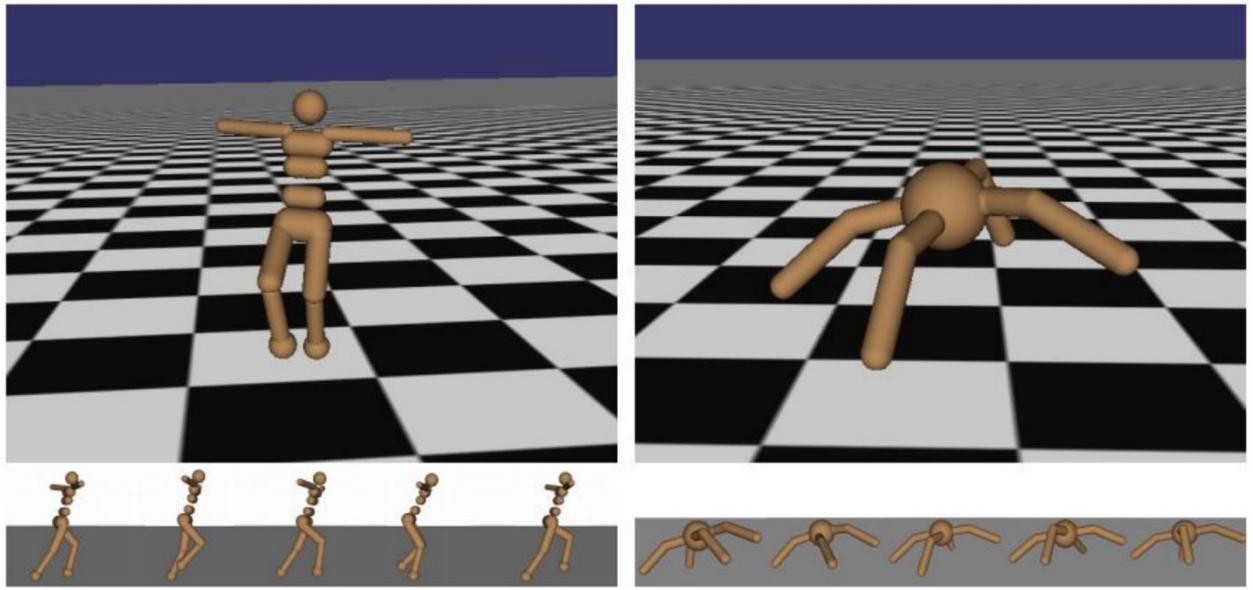
Objective: Win the board game Go.

State: Position of all pieces

Action: Where to put the next piece

Reward: 1 point if win at the end of the game, 0 otherwise

Example: Robot locomotion



Objective: teach the robot to walk forward

State: the current angle and position of the robot's joints (like knees, hips, etc)

Action: how much force or torque the robot applies to move each joint

Reward: the robot earns 1 point every moment it stays upright and moves forward

Example: Atari Games

Objective: teach the agent to play and win the Atari game

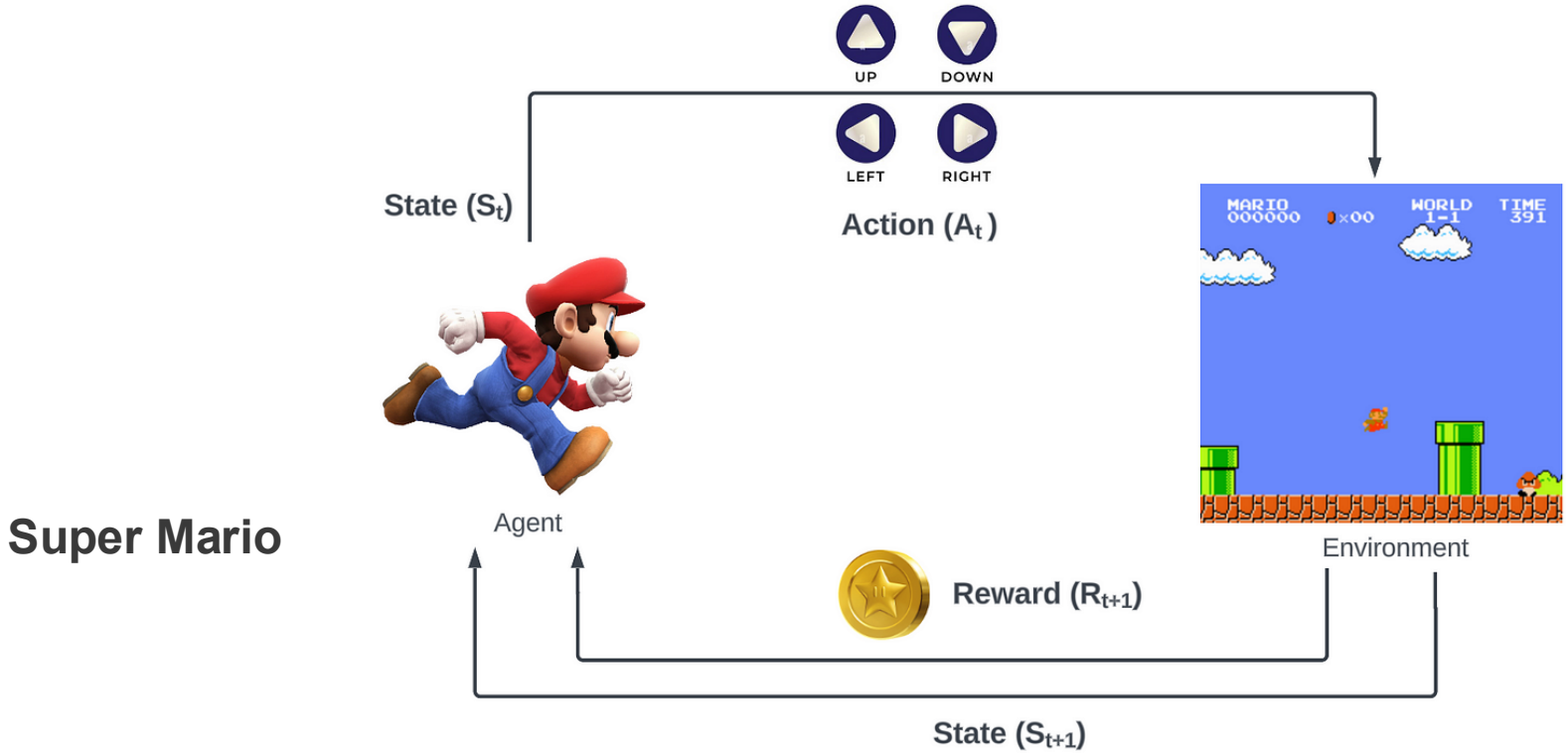
State: the current image on the screen (what the agent sees)

Action: Pressing buttons on the controller (or move left, right, jump, or shoot)

Reward: Points scored in the game, for example, hitting a target or collecting a coin gives a positive reward

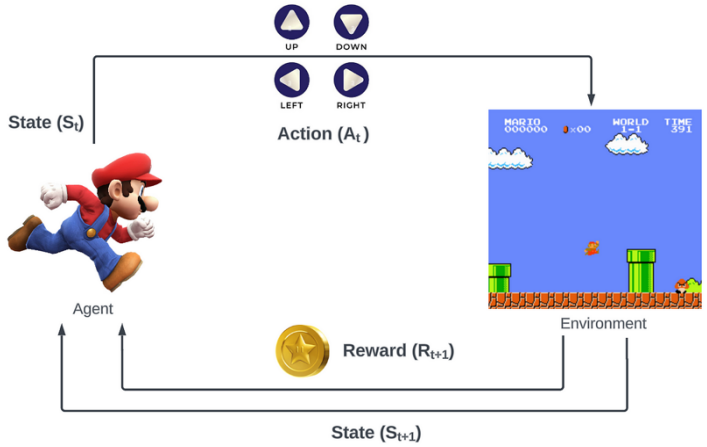
Agent and Environment

- **Agent**: the decision-maker that learns how to act
- **Environment**: everything the agent interacts with, or the system that the agent tries to control



State, Action, and Reward

- **State**: a representation of the environment at a specific moment
- **Action**: a move or decision the agent makes at a given state
- **Reward**: a numerical signal indicating the success of an action



- The agent observes the current state (S_t) of the environment and select actions (A_t)
- The environment updates based on the action and returns a new state (S_{t+1}) and reward (R_{t+1})
- Rewards are used to guide the agent's learning: actions that lead to higher rewards are reinforced (strengthened) over time, while less rewarding actions are discouraged

Return and Objective

Return: the total accumulated reward over time

The agent's **goal** is to maximize the expected return

Discounted return: future rewards may be worth less than immediate rewards

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0} \gamma^k R_{t+k+1}$$

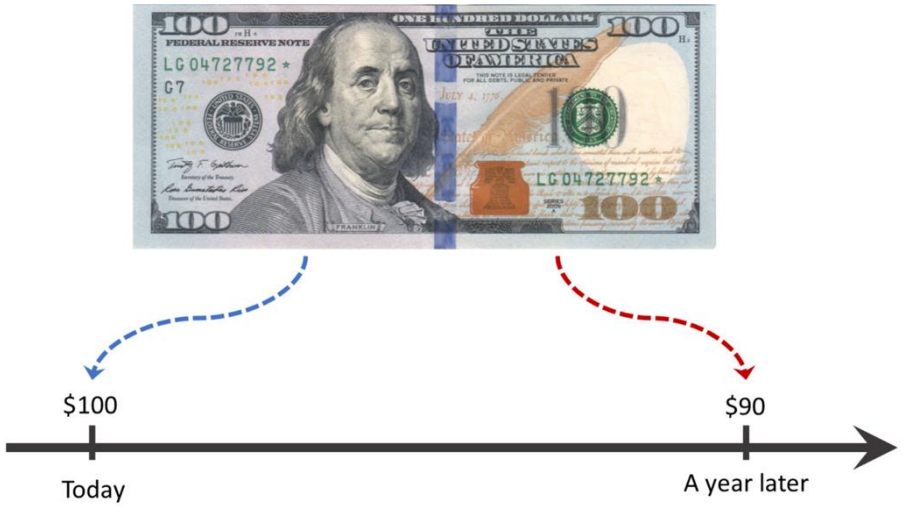
Objective function: often defined as the expected discounted return

Discount factor (γ)

- γ is the discount factor in range [0, 1]
- It indicates how much the future rewards are "worth" at the current moment (time t)

- If γ =0, we do not care about future rewards; the return is just the immediate reward → the agent is short-sighted

- If γ =1, the return is the unweighted sum of all subsequent rewards

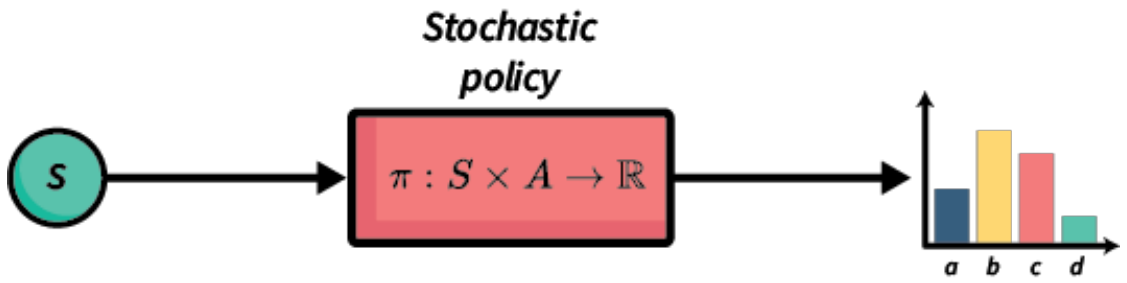


Due to inflation, earning a $100 bill right now could be worth more than earning it in the future

Policy

- The strategy to take actions base on the current states
  - **Deterministic policy**: always selects the same action for a state
  - **Stochastic policy**: selects actions based on probabilities
- The policy is what the agent learns and improves over time

$$\pi(a|s) \stackrel{\text{def}}{=} P[A_t = a|S_t = s]$$



Usually start from a random policy → find the optimal policy through learning $\pi_*(a|s)$

Value functions

Evaluates the expected return (i.e., cumulative future rewards) from a given state or state-action pair, under a particular policy π

- **State-value function $v_\pi(s)$**
- **Action-value function $q_\pi(s, a)$**

For example:

At each square of the maze, the mouse may think: *Hemm, this spot is worth something --- not because there's cheese there, but because it leads to cheese.*

- It evaluates how good is the current situation.
- It helps decide where to go next.

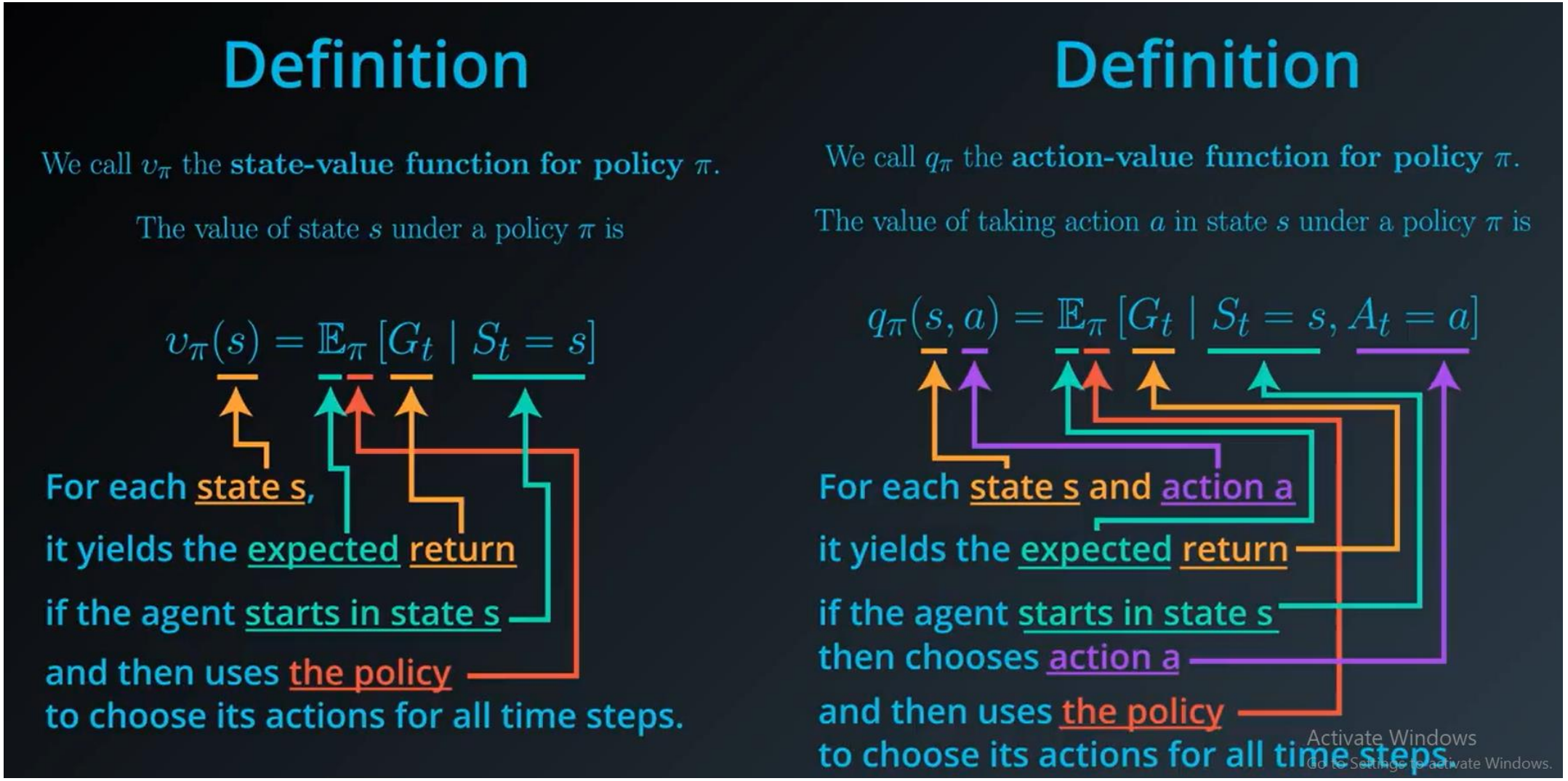
•Value functions help estimate long-term benefits.

•Example: In a chess game, choosing a move leads to different possible future states. Some states look more promising than others — the value function estimates how good each state is.

•Unlike reward, which is given only at the end (win/loss), the value function reflects the *potential* to win from a given state.

Two possible attacking moves for the black pawn: the white rook or the white bishop. They have different consequences which may lead to very different winning probability

Value functions



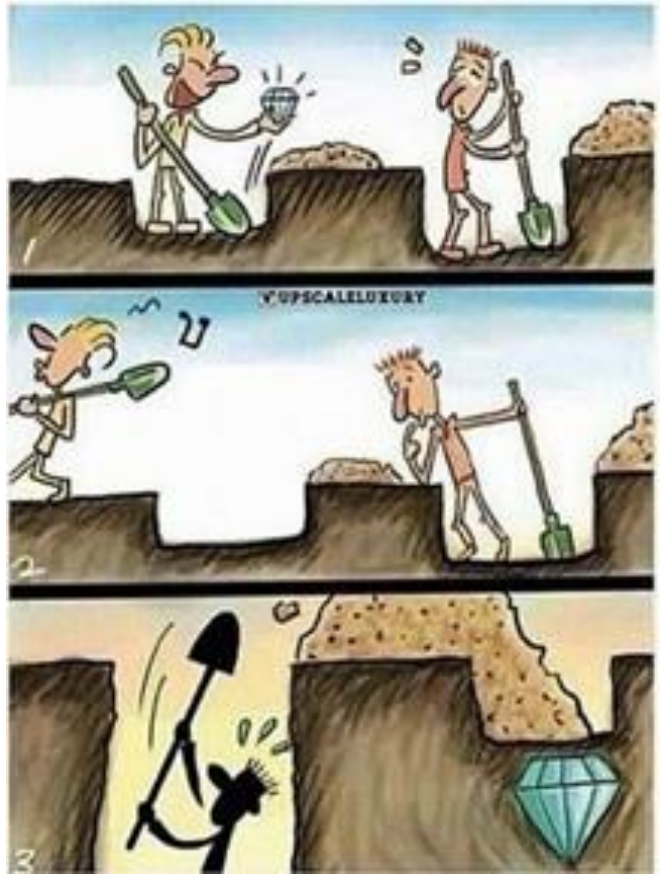
Exploration vs. Exploitation

Exploration: trying new actions to discover their effects

Exploitation: using known information to maximize reward

The agent must balance exploration and exploitation

Too much exploitation can miss better strategies; too much exploration slows learning



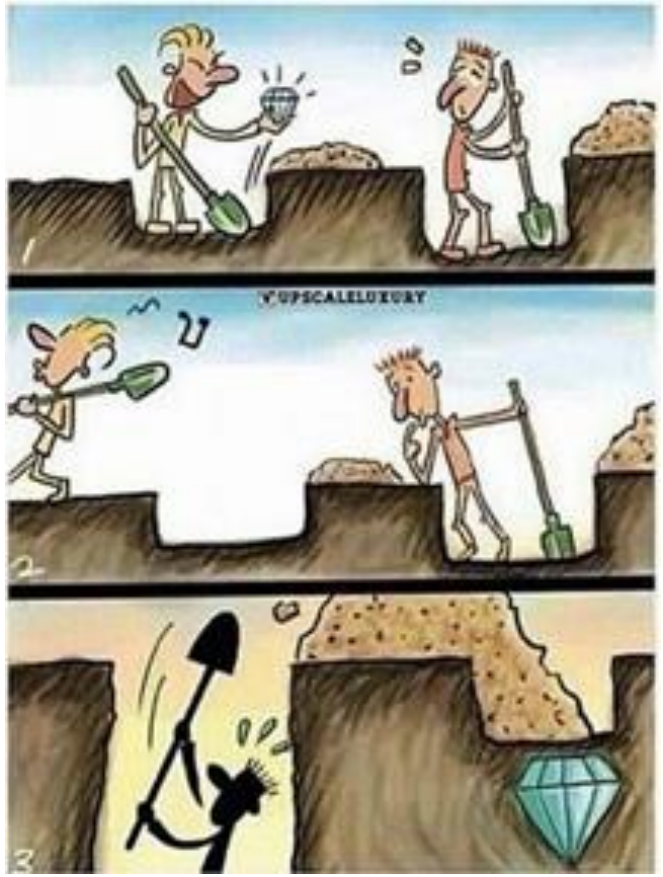
Exploration vs. Exploitation

If you only dig at the same spot where your friend found a diamond, you might get something — but you're relying on partial information. That spot worked once, but you don’t know if there are bigger or more diamonds elsewhere.

To find better rewards, you need to take the risk of exploring new areas. This is the essence of exploration vs. exploitation:

- **Exploitation** is digging where you already know there’s some reward.

- **Exploration** is venturing into the unknown, hoping to discover something even better



Markov Decision Process (MDP)

RL problems are typically modelled as MDPs, defined by the **transition probability**

$$P(s' \mid s, a)$$

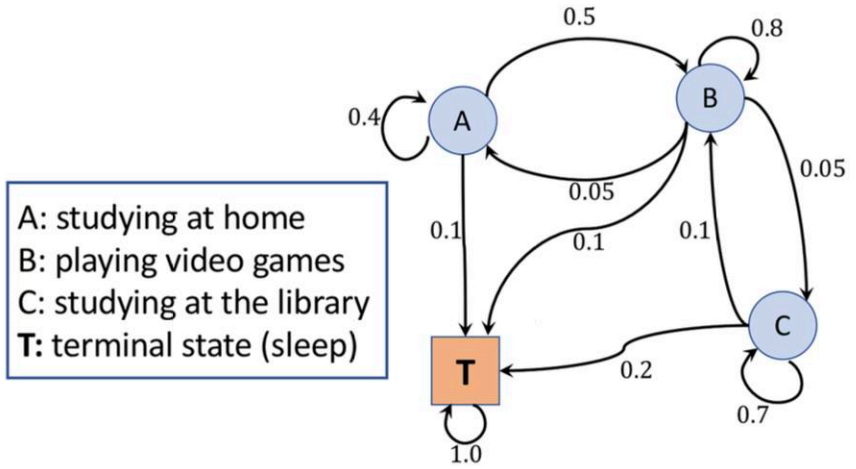
the probability of moving to a new state s', when taking action a from state s

Markov: The future depends only on the current state and action, not on the entire history

No memory on the past states/actions!

An example of Markov decision process

A student deciding between three different situations: A, B, C, near the final exam



Transition probabilities:

	A	B	C	T
A	0.4	0.5	0.0	0.1
B	0.05	0.8	0.05	0.1
C	0.0	0.1	0.7	0.2
T	0.0	0.0	0.0	1.0

MDPs: The transition probability only depends on the preceding time step

The sum of each row = 1

The decision are made every hour. After a few hours, it always ends at the **terminal state (T):** sleep
This is called an **episode.** A few examples:

$$\text{Episode 1: } BBCCCCBAT \rightarrow \text{pass (final reward} = +1)$$
$$\text{Episode 2: } ABBBBBBBBBBT \rightarrow \text{fail (final reward} = -1)$$
$$\text{Episode 3: } BCCCCCT \rightarrow \text{pass (final reward} = +1)$$

Learning from experience

The agent learns by interacting with the environment over many episodes

Experience: a sequence of (state, action, reward, next state) tuples

The agent updates its policy based on experience